

ECOCOLETA

Modelagem de um sistema PWA para informação sobre coleta de resíduos

ENTREGA 1 · Prática Extensionista IV

Integrante: João Eduardo Panissa.

Repositório do EcoColeta PWA: <https://github.com/m0biusz/EcoColeta-PWA>

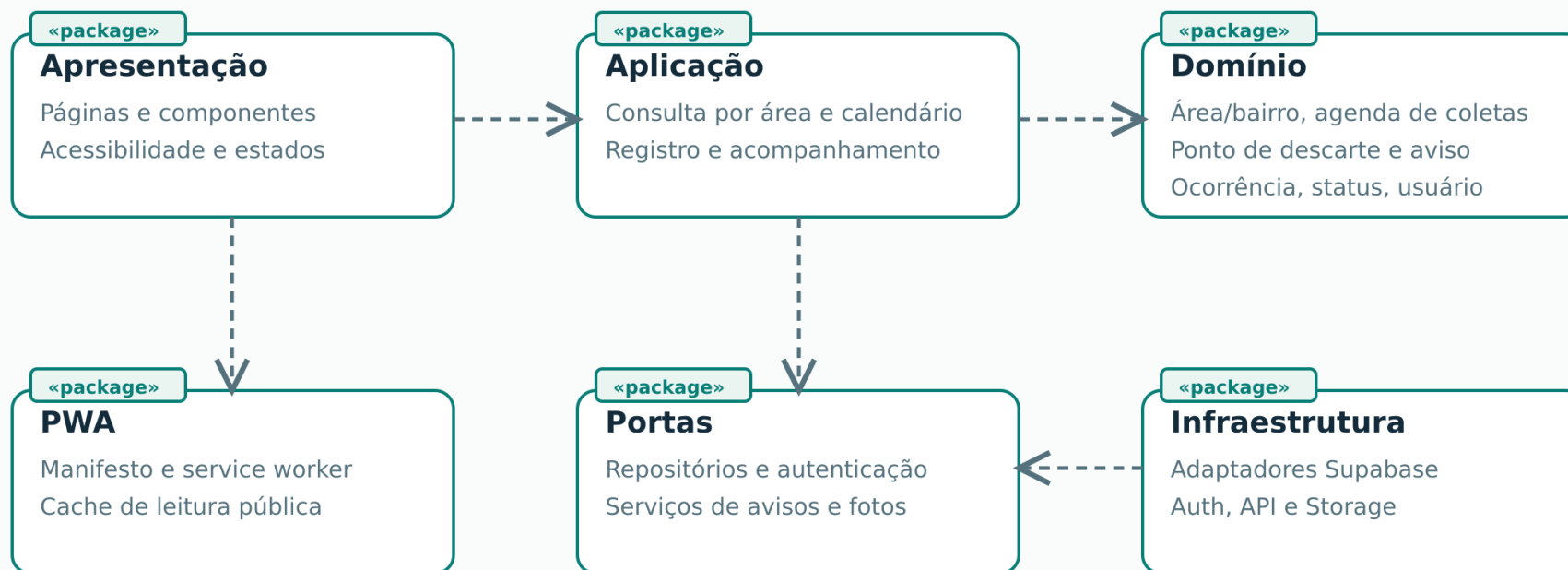
Escopo: arquitetura proposta para um PWA em repositório próprio, baseada no domínio do aplicativo Android anterior. Os serviços de nuvem e a aplicação web ainda dependem de implementação e ativação.

Base anterior: telas Android demonstrativas e requisitos de coletas, mapa, avisos e ocorrências. O PWA adapta esse escopo; não há dados municipais oficiais ou parceria confirmada.

1. Diagrama UML de pacotes

01 DIAGRAMA UML DE PACOTES

Dependências entre módulos previstos para o PWA



Dependência UML: linha tracejada e ponta aberta até o pacote usado.

Infraestrutura implementa as portas; o domínio não depende do provedor.

Pacotes com notação UML e dependências tracejadas. Apresentação chama casos de uso; a aplicação usa entidades e contratos; a infraestrutura implementa portas para comunicação externa. O módulo PWA organiza manifesto e cache.

2. Arquitetura de implantação

02 ARQUITETURA DE IMPLANTAÇÃO

Nós de execução, artefatos implantados e comunicação



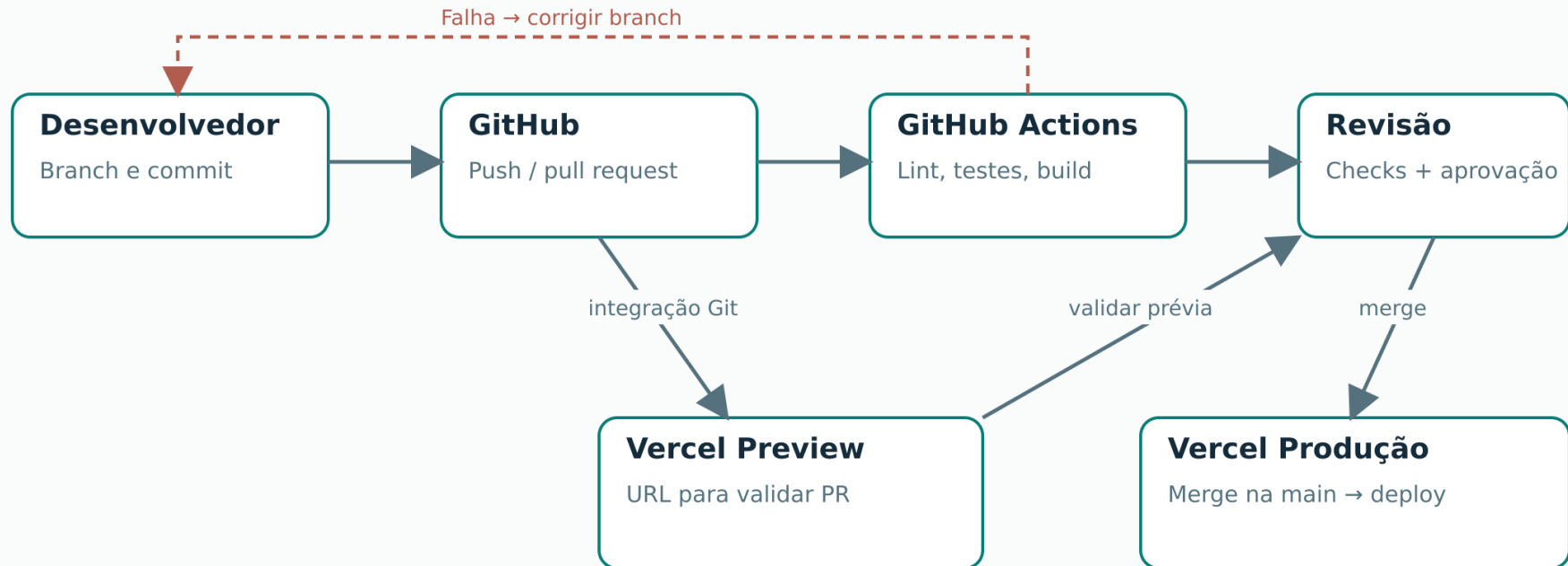
A PWA chama a API/Auth/Storage diretamente pelo navegador; Vercel entrega os arquivos da aplicação.
Sem internet: apenas dados públicos previamente armazenados; novos relatos exigem conexão.

O PWA executa no navegador. Vercel entrega o build estático; o navegador comunica-se com Auth, API e Storage do Supabase por HTTPS. O PostgreSQL só é acessado por serviços da plataforma, sob políticas RLS.

3. Arquitetura DevOps

03 ARQUITETURA DEVOPS

Fluxo planejado de integração, revisão e publicação



Falha nos checks → correção na branch → nova execução. O deploy depende de criar e conectar o projeto Vercel.

Alterações de banco: migrations versionadas, revisadas e aplicadas de modo controlado no Supabase.

Push e pull request disparam as verificações planejadas de CI. A integração Git com Vercel produz preview para revisão; após merge em main, publica o build de produção. Falhas retornam para correção. Fluxo ainda não ativado.

4. Infraestrutura de publicação

Escolha: Vercel (distribuição do PWA), Supabase Cloud (autenticação, API, PostgreSQL e Storage) e GitHub Actions (validação do código). O repositório GitHub está ativo; os projetos Vercel e Supabase ainda precisam ser criados e verificados.

Justificativa comparativa

Vercel + Supabase	Publicação a partir do Git e previews; backend gerenciado e banco relacional adequados a bairros, horários e ocorrências.
Netlify + Supabase	Também atende à hospedagem web. A escolha pela Vercel mantém uma integração única e documentada para o fluxo proposto.
Firebase	Alternativa gerenciada, mas a modelagem relacional proposta aproveita diretamente PostgreSQL no Supabase.
AWS / Azure	Ampla flexibilidade, com maior configuração operacional para uma primeira versão acadêmica.
Servidor próprio	Exigiria administrar servidor, TLS, atualizações, cópias de segurança e disponibilidade.

Provisionamento e cuidados

Pendência do item 5: criar os projetos Supabase e Vercel, configurar RLS, Storage e variáveis publicáveis e registrar evidência de ativação. Chaves privilegiadas não podem ir para o navegador. Testar preview e produção quando houver build do PWA.

Referências: MDN PWA · vercel.com/docs/git · supabase.com/docs/guides/database/postgres/row-level-security
supabase.com/docs/guides/getting-started/api-keys · docs.github.com/en/actions/tutorials/create-an-example-workflow

Documentação detalhada e fontes completas: [docs/pratica-extensionista-iv/infraestrutura.md](#)